Security in Software Applications

Information flow

SAPIENZA
UNIVERSITÀ DI ROMA

Daniele Friolo

friolo@di.uniroma1.it

https://danielefriolo.github.io

Motivating example

- Imagine using your mobile phone to
  1. locate nearest hotel using google
  2. book a room with your credit card
- Sensitive information
  - location information
  - credit card no
- (Un)wanted information flow
  - location may be leaked to google *only*
  - credit card info may be leaked to hotel *only*

Information Flow

- An interesting category of security requirements is about information flow
  - no confidential information should leak over network
  - no untrusted input from network should leak into database
- Information flow properties can be about confidentiality or integrity
- Note the difference with access control:
  - access control is about *access only*
  - information flow is *also* about *what you do with data once you accessed it*

Example Information Flow - Confidentiality

```
String hi; // security label secret
String lo; // security label public
```

Which program fragments (may) cause problems if **hi** has to be kept confidential?

```
1. hi = lo;
2. lo = hi;
3. lo = "1234";
4. hi = "1234";
```

```
5. println(lo);
6. println(hi);
7. readln(lo);
8. readln(hi);
```

Example Information Flow - Confidentiality

```
String hi; // security label secret
String lo; // security label public
```

Which program fragments (may) cause problems if **hi** has to be kept confidential?

✓ 1. `hi = lo;`
✗ 2. `lo = hi;`
✓ 3. `lo = "1234";`
? 4. `hi = "1234";`

✓ 5. `println(lo)`
✗ 6. `println(hi);`
✓ 7. `readln(lo);`
? 8. `readln(hi);`

Example Information Flow - Integrity

```
String hi; // high integrity (trusted) data
String lo; // low integrity (untrusted) data
```

Which program fragments (may) cause problems if integrity of hi is important ?

```
1. hi = lo;
2. lo = hi;
3. lo = "1234";
4. hi = "1234";
```

```
5. println(lo);
6. println(hi);
7. readln(lo);
8. readln(hi);
```

Example Information Flow - Integrity

```
String hi; // high integrity (trusted) data
String lo; // low integrity (untrusted) data
```

Which program fragments (may) cause problems if integrity of hi is important ?

✗ `1. hi = lo;`
✓ `2. lo = hi;`
✓ `3. lo = "1234";`
✓ `4. hi = "1234";`
✓ `5. println(lo);`
✓ `6. println(hi);`
✓ `7. readln(lo);`
✗ `8. readln(hi);`

Some points to note

- Beware: `readln(confidential)` and `println(high_integrity)` are *not* a problem
- Integrity and confidentiality are similarly modelled
  - Traditionally, most work on information flow looks at confidentiality; if we "flip" everything, we get a corresponding result for integrity
- Information flow properties are about ruling out unwanted influences/dependencies/interference/observations
- Note the difference between data flow properties and visibility modifiers (eg public, private) or, more generally, access control
  - it is not (just) about accessing data, but also about what you do with it

Questions

- how can we specify information flow policies?
- how can we enforce or check them?
  - dynamically or statically
- what is the semantics of information flow?
  - i.e. when can we say there is information flow? how can we characterize it?

Type-based information flow

- Type systems have been proposed as way to restrict information flow.

- Practical problem: often very (too) restrictive, because of trickier examples (next slides).

- The most mature realisation of a type system for information flow is probably in the Jflow/Jif system for Java by Andrew Myers & co.

Trickier examples for confidentiality

```
int hi; // security label secret
int lo; // security label public
```

Which program fragments (may) cause problems for confidentiality?

```
1. if (hi > 0) { lo = 99; }
2. if (lo > 0) { hi = 66; }
3. if (hi > 0) { print(lo);}
4. if (lo > 0) { print(hi);}
```

Trickier examples for confidentiality

```
int hi; // security label secret
int lo; // security label public
```

Which program fragments (may) cause problems for confidentiality?

✗ `1. if (hi > 0) { lo = 99; }`
✓ `2. if (lo > 0) { hi = 66; }`
✗ `3. if (hi > 0) { print(lo);}`
✗ `4. if (lo > 0) { print(hi);}`

implicit
aka
indirect flows

indirect vs direct flows

- Direct a.k.a. explicit flows happen by a "direct" assignment/leaking, e.g. as in

```
lo=hi;
```

or

```
println(hi);
```

More subtle forms of information flows:

1) indirect/implicit flows, e.g. by

```
if (hi > 0} { lo = 99; }
```

2) information flow via (non)termination or exceptions, or other hidden or covert channels

Trickier examples for confidentiality

Example

```
int hi; // security label secret
int lo; // security label public
```

Which program fragments (may) cause problems for confidentiality?

- `while (hi>99) do {....};`
- `while (lo>99) do {....};`
- `a[hi] = 23; // where a is high/secret`
- `a[hi] = 23; // where a is low/public`
- `a[lo] = 23; // where a is high/secret`
- `a[lo] = 23; // where a is low/public`

Trickier examples for confidentiality

```
int hi; // security label secret
int lo; // security label public
```

✗ 1. `while (hi>99) do {....};`
// timing or termination may reveal if hi > 99

✓ 2. `while (lo>99) do {....};` // no problem

✗ 3. `a[hi] = 23; // where a is high/secret`
// exception may reveal if hi is negative

✗ 4. `a[hi] = 23; // where a is low/public`
// contents of a may reveal value of hi and, again,
// exception may reveal if hi is negative

✗ 5. `a[lo] = 23; // where a is high/secret`
// exception may reveal the length of a, which may be secret

✓ 6. `a[lo] = 23; // where a is low/public`

Hidden channels

More subtle forms of indirect information flows can arise via hidden or covert channels, eg

- (non)termination

  eg **`while (hi>99) do {....};`**

  or **`if (hi=99) then {"loop"} else {"terminate"}`**

- execution time

  eg **`for (i=0; i<hi; i++) {...};`**

  or **`if (hi=1234) then {...} else {...}`**

- exceptions

  eg **`a[i] = 23`** may reveal length of a (if i is known), or leak info about **`i`** (if length of a is known), or reveal if a is null..

Questions

- how can we specify information flow policies?
- how can we enforce or check them?
  - dynamically or statically
    - a type system for information flow
- what do these policies really mean?
  - and is our enforcement sound?

Types for information flow (confidentiality)

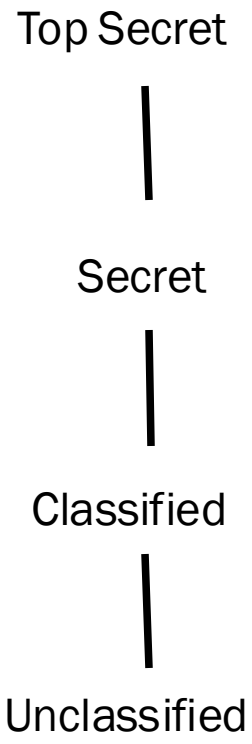
- We consider a lattice of different security levels

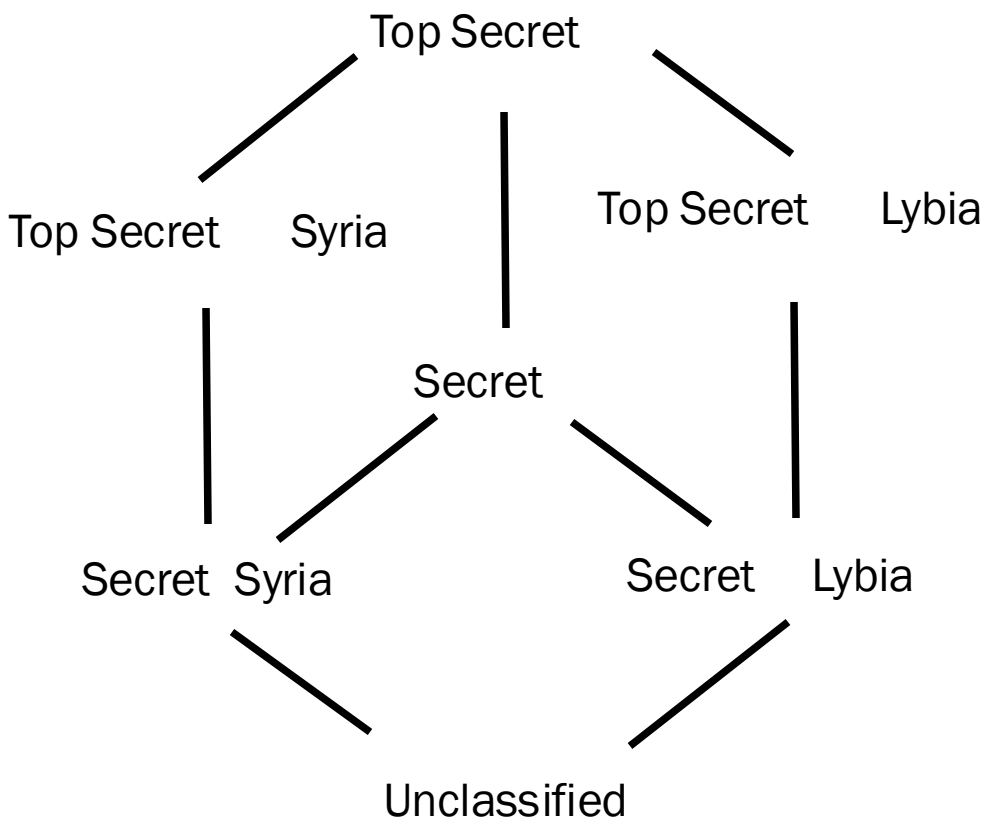
- For simplicity, just two levels
  - H(igh) or confidential, secret or
  - L(ow) public

- Typing judgements $\boldsymbol{\gamma} \vdash e : t$
  meaning e has type t in context $\boldsymbol{\gamma}$

- Context $\gamma = x_1 : t_1, \ldots x_n : t_n$ gives levels of program variables

More complex lattices





Rules for expressions

$\gamma \vdash e : t$ meaning e contains information of level t or lower

- variable $\gamma \vdash x : t$ if $\gamma(x) = t$
- operations $\dfrac{\gamma \vdash e : t \qquad \gamma \vdash e' : t}{\gamma \vdash e + e' : t}$ for binary operation + similar for n-ary
- subtyping $\dfrac{\gamma \vdash e : t \qquad t \leq t'}{\gamma \vdash e : t'}$

  where $L \leq H$

Rules for commands

γ ⊢ s : cmd t meaning s only writes to level t or higher

- assignment

$$\frac{\gamma \vdash e : t \qquad \gamma \vdash x : t}{\gamma \vdash x := e \ : \mathrm{cmd}\ t}$$

- if-then-else

$$\frac{\gamma \vdash e : t \qquad \gamma \vdash c1 : \mathrm{cmd}\ t \qquad \gamma \vdash c2 : \mathrm{cmd}\ t}{\gamma \vdash \mathrm{if}\ e\ \mathrm{then}\ c1\ \mathrm{else}\ c2 : \mathrm{cmd}\ t}$$

- subtyping

$$\frac{\gamma \vdash c : cmd\ t \qquad \mathrm{cmd}\ t \leq \mathrm{cmd}\ t'}{\gamma \vdash c : c : \mathrm{cmd}\ t'}$$

where ~~$cmd\ t \leq cmd\ t'$~~ iff $t \geq t'$ (anti-monotonicity)

Rules for commands

γ ⊢ s : cmd t meaning s only writes to level t or higher

- a

- i

$$\gamma \vdash \text{if } e \text{ then } c1 \text{ else } c2 : \text{cmd } t \qquad t$$

Note: commands sybtyping backwards from expression subtyping rule!

- In expressions, can replace H with L without causing an insecure read up
- In commands, can replace cmd L with cmd H without causing an insecure write down

- subtyping

$$\frac{\gamma \vdash c : cmd\ t \qquad \text{cmd } t \leq \text{cmd } t'}{\gamma \vdash c : c\text{: cmd } t'}$$

where $cmd\ t \leq cmd\ t'$ iff $t \geq t'$ (anti-monotonicity)

Rules for commands

$\gamma \vdash s : \text{cmd}\ t$ meaning s only writes to level t or higher

- composition

$$\frac{\gamma \vdash c1 : \text{cmd}\ t \qquad \gamma \vdash c2 : \text{cmd}\ t}{\gamma \vdash c1; c2 : \text{cmd}\ t}$$

- while

$$\frac{\gamma \vdash e : t \qquad \gamma \vdash c : \text{cmd}\ t}{\gamma \vdash \text{while}\ e\ \text{do}\ c : \text{cmd}\ t}$$

Soundness and Completeness

- *Soundness* of the type system:
  programs that are well-typed do no leak
- *Completeness* of the type system:
  programs that do not leak can be typed

Is the type system on preceding slides

- *sound?*
- *complete?*

How can we determine this?

Counterexamples for completeness

It is easy to give examples that are not typable but do not leak, eg

- `if (false) then { lo = hi; }`
- `lo = hi + 1 - hi;`
- `lo = hi; lo = 12;`

Counterexamples for completeness

- Is this type system sound?
  - i.e. does it prevent the information flows that we want to prevent
- How do we define what we want to prevent?

- This is commonly done by *non-interference* properties, which try to capture the notion of what can be observed

In other words, non-interference gives a precise semantics for what “information flow” means

Soundness wrt non-interference

Definition For memories (or program states) μ and ν, we write

$\mu \sim_L \nu$ iff μ and ν agree on low variables.

Definition (Non-interference)

A program C does not leak information if, for all $\mu \sim_L \nu$:

if executing C in μ terminates and results in μ',

and executing C in ν terminates and results in ν',

then $\mu' \sim_L \nu'$

Theorem (Soundness)

if $\gamma \vdash C : \text{cmd t}$ then C does not leak information

Termination as covert channel?

Definition (Non-interference) **termination-*in*sensitive**

A program C does not leak information if, for all $\mu \sim_L \nu$:

if executing C in μ terminates and results in μ', and

executing C in ν terminates and results in ν', then $\mu' \sim_L \nu'$

Does this rule out (non) termination as hidden channel
(as observation to distinguish two runs)?

Definition (Termination-sensitive non-interference)

A program C does not leak information if, for all $\mu \sim_L \nu$:

if executing C in μ terminates in μ', then executing C in ν also terminates,
and results in some ν' with $\mu' \sim_L \nu'$

It means that

- **either both terminate or both diverge**, and
- if they terminate, the low parts match

While-rule for termination-sensitive non- interference

The while-rule

$$\frac{\gamma \vdash e : t \qquad \gamma \vdash c : \mathrm{cmd}\ t}{\gamma \vdash \text{while } e \text{ do } c : \mathrm{cmd}\ t}$$

does *not* rule out non-termination as covert channel

A more restrictive rule

$$\frac{\gamma \vdash e : L \qquad \gamma \vdash c : \mathrm{cmd}\ L}{\gamma \vdash \text{while } e \text{ do } c : \mathrm{cmd}\ L}$$

does rule this out.

(How? NB this is very restrictive!)

- A similar change needed for if-then-else rule.

Bell-La Paluda

- The type system is similar to classic Bell-La Paluda system which combines
  - Mandatory Access control (MAC)
  - Multi-Level Security (MLS)

  and protects information flow between files by rules
  - no read up
  - no write down
    - *nb similarity with our typing rules*
- But for program accessing variables, instead of a process accessing files
- Moreover: type system checks program statically, whereas Bell-LaPadula monitors process at runtime

Other notions of secure information flow

Other definitions of what it means to be secure (in the sense of non-leaking) are needed if

- if programs can throw exceptions
  - exceptions are another covert channel, just like non-termination

- if programs are multi-threaded or non-determinisitic
  - because execution of a program can then result in several outcomes
    - multi-threaded programs are non-deterministic, because results can depend on scheduling

Information flow for non-deterministic programs

Definition (Possibilistic NI)

A non-deterministic program **C** does not leak information if for all $\mu \sim_L \nu$ if executing C in μ terminates in μ', then executing C in ν *can* terminate in some ν' with $\mu' \sim_L \nu'$

This still ignores *probabilistic* information flows, for which one would take the chance that C does terminate in some ν' with $\mu' \sim_L \nu'$ into account

– Running the program multiple times, you might be able to observe something, for example:

```
If hi == 0: low = 0 w.p. 0.9 and 1 w.p 0.1
If hi == 1: low = 1 w.p. 0.1 and 1 w.p 0.9
```

- Observation of low does not leak information in a single execution since 0 and 1 are both possible outputs of low
- In multiple executions the adversary can inspect the distribution of low

The problem with secure information flow

- *Practical* problem with secure information flow: the extreme restrictions it imposes, esp. when it come to ruling out implicit flows
  - eg no while loop with a high guard
  - note that login program leaks information about the password

- More generally, some way of allowing forms of declassification is needed in practice

Declassification

More *permissive* forms of information flow can allow de-classification, eg

- for *confidentiality*:
  - output of encryption operation is labelled as public, even though it depends on secret data.
- for *integrity*:
  - output of input validation routine may be trusted, even though it depends on untrusted data
  - output of routine that checks digital signature may be trusted, even though it depends on untrusted data

Information Flow in "practice"

- Static enforcement
  - JFlow/Jif is a type system for expressing & enforcing information flow properties for Java
    - using it requires serious effort
  - Many code analysis tools perform some data flow analysis
    - Eg to spot SQL injection problems
    - Recall PREfast did this, but only intra-procedural
    - NB typically for integrity, not confidentiality
    - Often unsound/incomplete, as concession to practicality
- Dynamic enforcement
  - Perl has an *runtime monitoring* of information flow properties (again for integrity properties) using tainting

Pragmatic approaches typically worry less – if at all - about implicit flows.

- Indeed, are implicit flows an issue for integrity?

Summary

- What is information flow?
  - explicit flows
  - implicit flows
  - covert channels
- How can we *syntactically* control information flow, using type systems?

(at compile time, ie. statically)

- How can we *semantically* define information flow?
  - non-interference
- Termination sensitive vs termination insensitive